

Week 1

What is a Database?

Definition: A database is a structured collection of data that can be accessed, managed, and updated. It serves as a repository for organized data that supports efficient retrieval and management.

Examples:

A customer directory in an e-commerce application.

A hospital's patient records system.

An educational institution's student database.

What is a Database?

Characteristics of Databases:

1

- Organized and structured.

2

- Persistent and stored electronically.

3

- Accessible by multiple users or applications.

What is a Database Management System (DBMS)?

❑ **Definition:** A Database Management System (DBMS) is software designed to define, manipulate, retrieve, and manage data in a database.

❑ **Core Functions of DBMS:**

❑ **Data Storage:** Efficiently stores large amounts of data.

❑ **Data Retrieval:** Allows quick access to data using queries.

❑ **Data Manipulation:** Supports operations like insertion, deletion, and updating of records.

❑ **Data Security:** Ensures authorized access and protects data integrity.

Popular DBMS Examples:

- Relational DBMS: MySQL, PostgreSQL, Oracle Database.
- NoSQL DBMS: MongoDB, Cassandra, DynamoDB.
- Cloud DBMS: Google Cloud Spanner, Amazon RDS.

Key Differences Between Data and Databases

Aspect	Data	Database
Definition	Raw, unprocessed facts and figures.	Organized collection of structured data.
Example	"John, 25"	A table with columns: Name, Age.
Purpose	Basic unit of information.	Stores and manages related data.

Components of a DBMS

- **Database Engine:** Core software component that performs storage and retrieval tasks.
- **Query Processor:** Interprets user queries (e.g., SQL commands) and converts them into actionable tasks.
- **Data Definition Language (DDL):** Defines database schema and structure (e.g., CREATE, ALTER).
- **Data Manipulation Language (DML):** Performs data operations like insertion, deletion, and updates (e.g., INSERT, UPDATE, DELETE).
- **Transaction Manager:** Ensures data integrity and consistency during concurrent user access.

Types of DBMS

- **Relational DBMS (RDBMS):**
 - Data is organized in tables with rows and columns.
 - Follows a strict schema and supports SQL.
 - Example: MySQL, PostgreSQL.
- **NoSQL DBMS:**
 - Designed for unstructured or semi-structured data.
 - Uses document, key-value, or graph data models.
 - Example: MongoDB, Redis.

Types of DBMS(Cont.)

- **Hierarchical DBMS:**
 - Data is organized in a tree-like structure.
 - Example: IBM's IMS.
- **Network DBMS:**
 - Data is organized in a graph, with multiple parent-child relationships.
 - Example: Integrated Data Store (IDS).
- **Object-Oriented DBMS (OODBMS):**
 - Supports object-oriented data models.
 - Example: ObjectDB.

ADVANTAGES OF USING DBMS

◉ **Data Centralization:**

- Provides a single location for data storage, reducing redundancy.

◉ **Data Integrity and Consistency:**

- Enforces rules to ensure data remains accurate and consistent.

◉ **Data Security:**

- Implements user authentication and access controls.

ADVANTAGES OF USING DBMS

◎ Backup and Recovery:

- Ensures data is protected and can be restored in case of failure.

◎ Concurrent Access:

- Supports multiple users accessing the database simultaneously without conflicts.

Real-World Applications of Databases

- **Banking:**
 - Customer accounts, transactions, and loan processing.
- **Healthcare:**
 - Patient records, prescriptions, and billing.
- **Retail:**
 - Inventory management, customer orders, and sales analytics.
- **Education:**
 - Student management, course registrations, and grading systems.
- **Social Media:**
 - User profiles, posts, and interactions.

Evolution of Database Systems

- **Early Systems:**
 - Flat-file systems and manual data storage techniques.
- **Hierarchical and Network Models:**
 - Emerged in the 1960s to handle more complex data relationships.
- **Relational Databases:**
 - Introduced by E.F. Codd in the 1970s; became the standard for structured data.
- **NoSQL and Cloud Databases:**
 - Developed in the 21st century to handle large-scale, unstructured data and distributed systems.

Week 2

Introduction to Data Models

- o **Definition:** A data model defines the structure, organization, and constraints of data stored in a database. It provides a framework for how data is stored, accessed, and manipulated.
- o **Purpose of Data Models:**
 - o To represent real-world entities and their relationships.
 - o To ensure data consistency and integrity.
 - o To facilitate the design and implementation of databases.

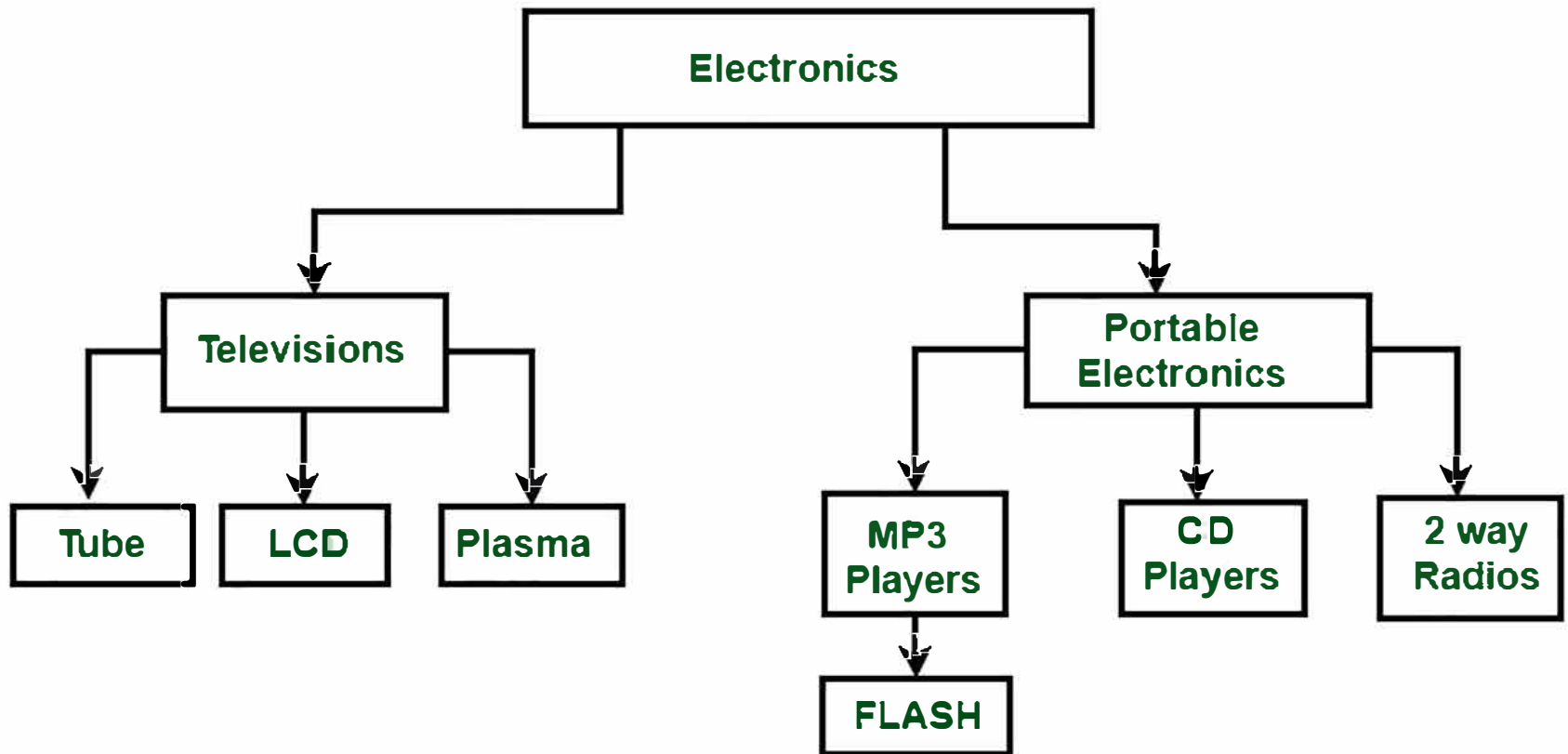
Components of a Data Model:

- **Structure:** Defines how data is organized (e.g., tables, hierarchies).
- **Operations:** Specifies the methods to manipulate data (e.g., insert, update).
- **Constraints:** Enforces rules to maintain data integrity.

TYPES OF DATA MODELS

Hierarchical Data Model

- ◉ **Definition:** Data is organized in a tree-like structure with parent-child relationships.
- ◉ **Characteristics:**
 - Each child node has only one parent node.
 - Relationships are represented as a hierarchy.
- ◉ **Advantages:**
 - Simple to understand and implement.
 - Fast data access for parent-child relationships.



Hierarchical Data Model

Types of Data Models(cont.)

o Disadvantages:

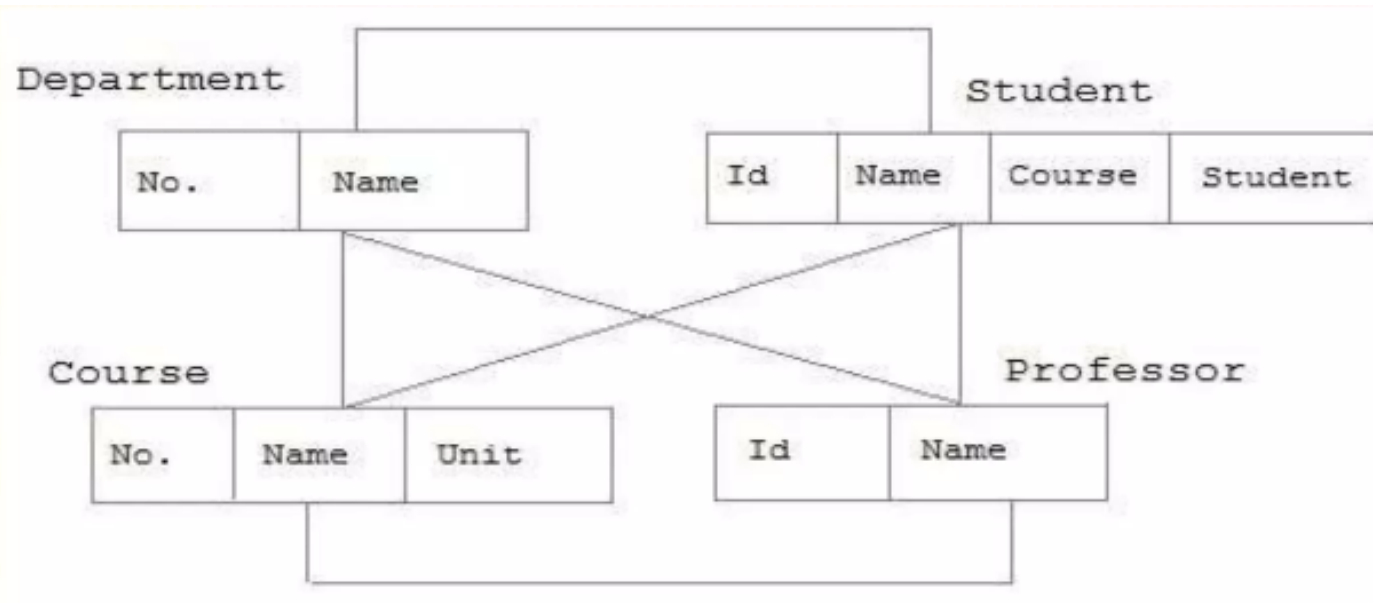
- o Rigid structure; difficult to handle many-to-many relationships.
- o Modifications require restructuring the hierarchy.

o Example:

- o An organizational structure where each employee reports to one manager.

Types of Data Models(cont.)

- **Network Data Model**
- **Definition:** Data is organized in a graph structure, allowing multiple parent-child relationships.
- **Characteristics:**
 - Data elements are connected by links, representing relationships.
 - More flexible than the hierarchical model.
- **Advantages:**
 - Supports many-to-many relationships.
 - Efficient data retrieval for complex relationships.



Network Data Model

Types of Data Models(cont.)

- **Disadvantages:**

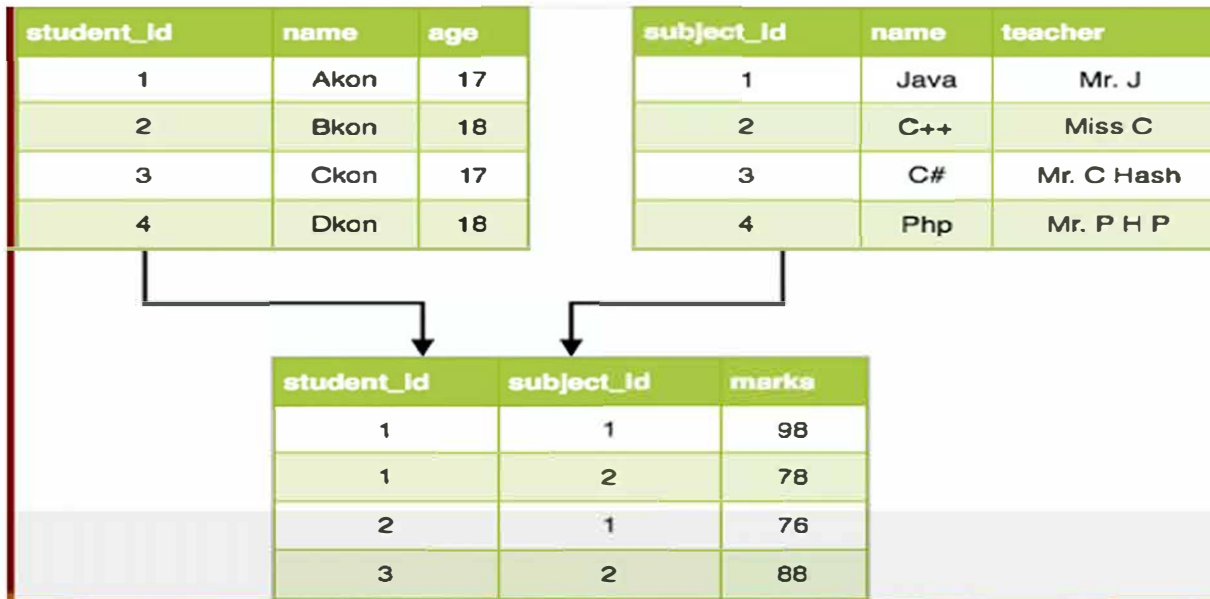
- Complex to design and maintain.
- Requires specialized knowledge to implement.

- **Example:**

- A flight reservation system where a flight can connect to multiple airports.

Types of Data Models(cont.)

- **Relational Data Model**
- **Definition:** Data is organized in tables (relations) with rows and columns.
- **Characteristics:**
 - Each table represents an entity, and each row represents an instance of the entity.
 - Relationships between tables are established using keys (primary and foreign).
- **Advantages:**
 - Simple and widely used.
 - Supports SQL for data manipulation.
 - Ensures data consistency and integrity.



Relational Data Model

Types of Data Models(cont.)

- **Disadvantages:**

- Performance issues with very large datasets.
- Limited for complex relationships like hierarchical or network structures.

- **Example:**

- A student database with tables for Students, Courses, and Enrollments.

TYPES OF DATA MODELS(CONT.)

- ◉ **Object-Oriented Data Model**

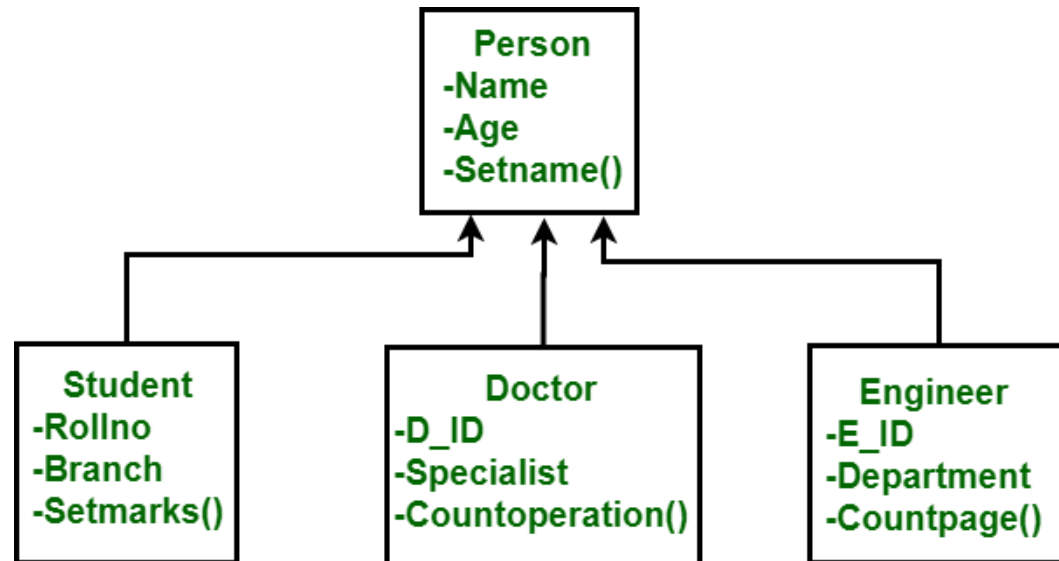
- ◉ **Definition:** Combines object-oriented programming concepts with database principles.

- ◉ **Characteristics:**

- Data is represented as objects with attributes and methods.
- Supports inheritance, encapsulation, and polymorphism.

- ◉ **Advantages:**

- Ideal for applications with complex data relationships.
- Supports multimedia data (images, videos).



Object-Oriented Data Model

Types of Data Models(cont.)

- **Disadvantages:**

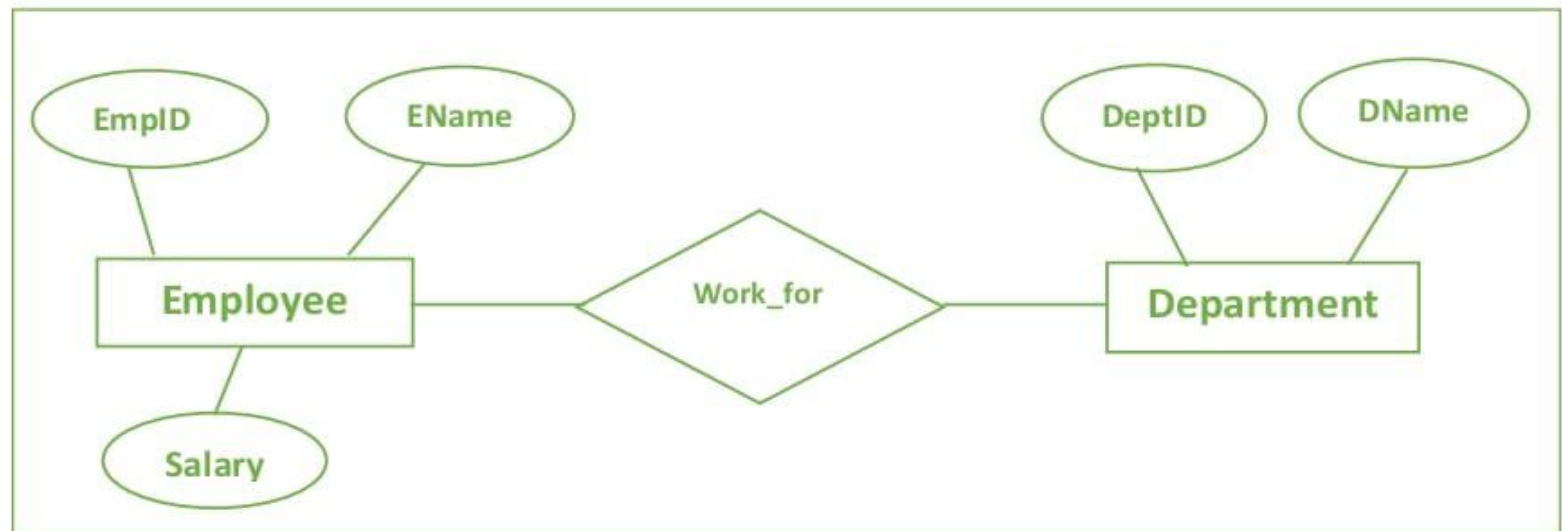
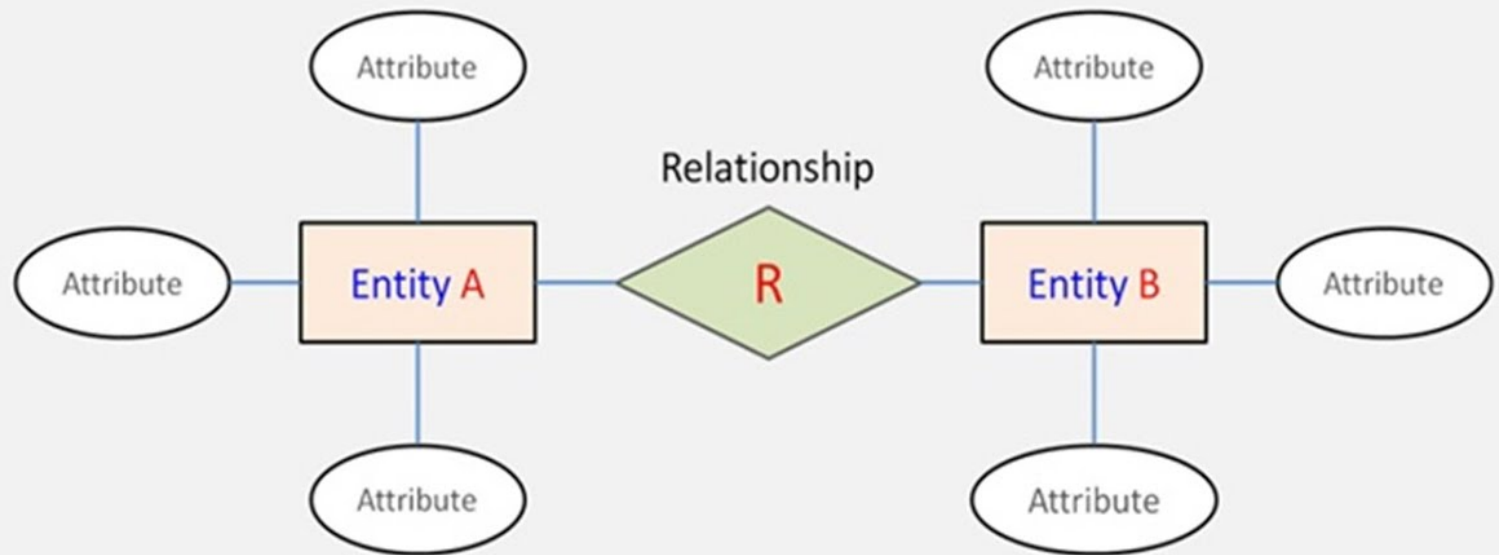
- Complex to implement and maintain.
- Less popular compared to the relational model.

- **Example:**

- A multimedia library where books, videos, and audio files are stored as objects.

Types of Data Models(cont.)

- **Entity-Relationship (ER) Model**
- **Definition:** Represents data as entities, attributes, and relationships between entities.
- **Characteristics:**
 - **Entities:** Objects or things in the real world (e.g., Student, Course).
 - **Attributes:** Properties of entities (e.g., Name, Age).
 - **Relationships:** Associations between entities (e.g., Enrolled).
- **Advantages:**
 - Easy to visualize data relationships using ER diagrams.
 - Useful for designing relational databases.



Types of Data Models(cont.)

- ▣ **Disadvantages:**

- Limited to high-level design; needs to be converted into a physical model.

- ▣ **Example:**

- An ER diagram showing Students enrolled in Courses taught by Professors.

Types of Data Models(cont.)

- **NoSQL Data Models**
- **Definition:** Designed for unstructured or semi-structured data, offering high scalability and flexibility.
- **Types of NoSQL Models:**
 - ▣ **Document Model:** Stores data in JSON or BSON format (e.g., MongoDB).
 - ▣ **Key-Value Model:** Data stored as key-value pairs (e.g., Redis).
 - ▣ **Column-Family Model:** Optimized for read/write operations in columns (e.g., Cassandra).
 - ▣ **Graph Model:** Focuses on relationships using nodes and edges (e.g., Neo4j).
- **Advantages:**
 - ▣ Highly scalable and suitable for big data.
 - ▣ Flexible schema design.

Person

name	phone
John	3634
Sue	6343

Orders

personName	date	product
John	2002	Gizmo
John	2004	Gadget
Sue	2002	Gadget

```
{ "Person":  
  [{ "name": "John",  
    "phone": 3646,  
    "Orders": [  
      { "date": 2002, "product": "Gizmo" },  
      { "date": 2004, "product": "Gadget" }  
    ]  
  },  
  { "name": "Sue",  
    "phone": 6343,  
    "Orders": [  
      { "date": 2002, "product": "Gadget" }  
    ]  
  }  
]
```

NoSQL Data Models

Types of Data Models(cont.)

❑ **Disadvantages:**

- ❑ Lacks standardization.
- ❑ Limited support for complex queries.

❑ **Example:**

- ❑ Social media platforms storing user interactions in a graph database.

Comparison of Data Models

Model	Structure	Use Case	Example
Hierarchical	Tree	Organizational chart	IBM IMS
Network	Graph	Flight reservation system	IDS
Relational	Tables	Enterprise applications	MySQL, Oracle
Object-Oriented	Objects	Multimedia or CAD applications	ObjectDB
ER Model	Entities and Relations	Database design	ER Diagrams
NoSQL	Document, Key-Value	Big data and real-time analytics	MongoDB, Cassandra

Week 3

Introduction to Database Design

- **Definition:** Database design is the process of defining the structure, storage, and organization of data in a database to ensure efficiency, scalability, and data integrity.
- **Objectives of Database Design:**
 - To minimize data redundancy.
 - To ensure data consistency and integrity.
 - To enhance data retrieval and manipulation efficiency.

Introduction to Database Design(cont.)

- **Phases of Database Design:**
- **Conceptual Design**
- **Logical Design**
- **Physical Design**

Phases of Database Design

- **Conceptual Design**
- **Definition:** Focuses on high-level representation of the data structure using Entity-Relationship (ER) diagrams.
- **Key Components:**
 - ▣ **Entities:** Objects or things in the real world (e.g., Customer, Product).
 - ▣ **Attributes:** Properties of entities (e.g., Customer Name, Product Price).
 - ▣ **Relationships:** Associations between entities (e.g., Customer purchases Product).

Phases of Database Design

- **Tools Used:**

- ER Diagrams

- UML Diagrams

- **Example:** An ER diagram representing Customers and Orders where a Customer can place multiple Orders.

Phases of Database Design

□ Logical Design

□ **Definition:** Converts the conceptual design into a logical model that can be implemented in a database management system (DBMS).

□ Steps Involved:

- Transform entities into tables.
- Define primary keys for each table.
- Define relationships using foreign keys.
- Normalize the database.

Phases of Database Design

- **Normalization:**

- Process of organizing data to minimize redundancy and improve integrity.

- **Normal Forms:**

- **First Normal Form (1NF):** Eliminate repeating groups; ensure atomic values.
- **Second Normal Form (2NF):** Eliminate partial dependencies; all non-key attributes depend on the whole primary key.
- **Third Normal Form (3NF):** Eliminate transitive dependencies; non-key attributes depend only on the primary key.

Phases of Database Design

- **Physical Design**
- **Definition:** Focuses on the physical storage and performance optimization of the database.
- **Key Considerations:**
 - ▣ **Indexing:** Create indexes on frequently queried columns for faster data retrieval.
 - ▣ **Partitioning:** Divide large tables into smaller, more manageable pieces.
 - ▣ **Storage Optimization:** Ensure efficient use of disk space.
- **Example:**
 - ▣ Adding an index on the "CustomerID" column to speed up searches.

Steps in Database Design

□ Requirement Analysis

- Gather and analyze the requirements of the database from stakeholders.
- Understand the data needs, use cases, and constraints.

□ Conceptual Modeling

- Use ER diagrams to visualize entities, attributes, and relationships.

□ Logical Modeling

- Convert the conceptual model into a relational schema.
- Normalize the schema to eliminate redundancy.

Steps in Database Design

□ **Physical Modeling**

- Define how the data will be stored on the disk.
- Optimize for performance and storage efficiency.

□ **Implementation**

- Use a DBMS to create the database schema and populate it with data.

□ **Testing and Validation**

- Verify that the database meets requirements and performs efficiently.

Key Principles of Good Database Design

- ❑ **Simplicity:** Keep the design as simple as possible while meeting requirements.
- ❑ **Scalability:** Ensure the database can handle growing amounts of data.
- ❑ **Integrity:** Enforce data integrity through constraints and normalization.
- ❑ **Flexibility:** Design for adaptability to future changes.
- ❑ **Performance:** Optimize for query speed and storage efficiency.

Challenges in Database Design

- ❑ Balancing normalization and performance.
- ❑ Handling complex relationships and large datasets.
- ❑ Addressing security and access control requirements.
- ❑ Accommodating changes in requirements.

Tools for Database Design

- **ERD Tools:**

- Lucidchart, draw.io, Microsoft Visio

- **DBMS Tools:**

- MySQL Workbench, Microsoft SQL Server Management Studio (SSMS), Oracle SQL Developer

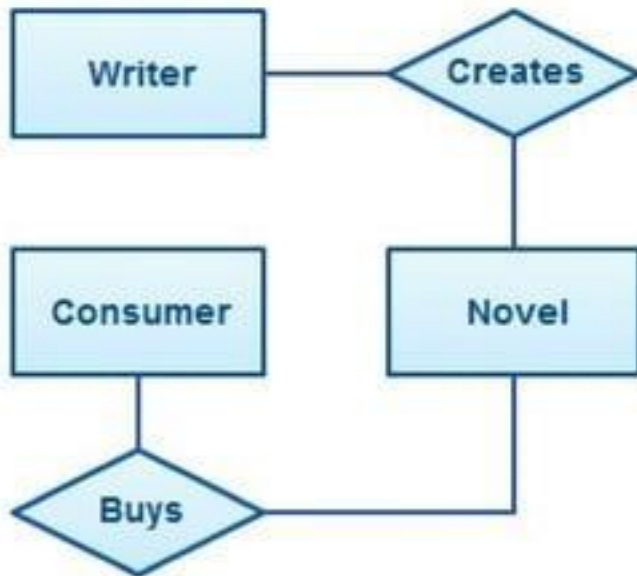
- **Other Tools:**

- dbdiagram.io, DbSchema, Navicat

Entity Relationship Model

Entity Relationship Model

- An Entity Relationship Diagram (ERD) is a *visual representation of different data using conventions that describe how these data are related to each other.*
- For example, the elements writer, novel, and consumer may be described using ER diagrams this way:



The elements inside **rectangles** are called entities while the items inside **diamonds** denote the relationships between entities.

Entity

- *An entity can be a real-world object*, either animate or inanimate, that can be *easily identifiable*.
 - For example, in a school database, **students**, **teachers**, **classes**, and **courses offered** can be considered as **entities**.
 - All these entities have some *attributes or properties* that give them their identity.
- An **entity set** is a collection of similar types of entities.
 - An entity set may contain *entities with attribute sharing similar values*.
 - For example,
 - a Students set may contain all the students of a school;
 - likewise a Teachers set may contain all the teachers of a school from all faculties. Entity sets need not be disjoint.

Weak Entity

- A weak entity is an entity that **depends on the existence of another entity.**
- In more technical terms it can be defined as an entity that cannot be identified by its own attributes.
- It uses a foreign key combined with its attributes to form the primary key.
- An entity like **order item** is a good example for this.
 - The order item will be meaningless without an order so it depends on the existence of order.



Attributes

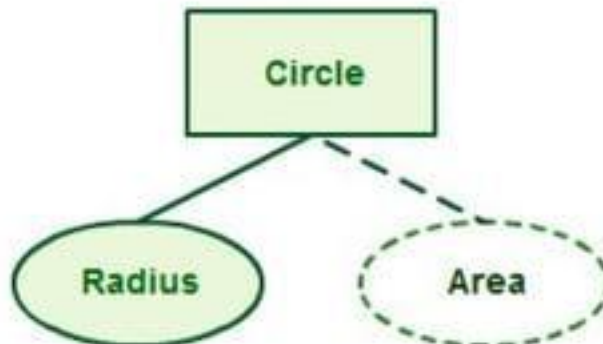
- Entities are represented by means of their *properties*, called **attributes**.
- All attributes have values.
- For example, a **student entity** may have **name, class,** and **age** as attributes.
- *There exists a domain or range of values that can be assigned to attributes.*
- For example, a **student's name** cannot be a numeric value. It has to be **alphabetic**. A **student's age cannot be negative**, etc.

Types of Attributes

- **Simple attribute** – Simple attributes are atomic values, which cannot be divided further. For example, **a student's phone number is an atomic value of 10 digits.**
- **Composite attribute** – Composite attributes are made of more than one simple attribute. For example, **a student's complete name may have first_name and last_name.**

Types of Attributes (contd.,)

- **Derived attribute** – Derived attributes are the attributes that do not exist in the physical database, but their **values are derived from other attributes** present in the database.
- For example, average_salary in a department should not be saved directly in the database, instead it can be derived.
- For another example, age can be derived from data_of_birth.
- For example for a circle the area can be derived from the radius.



Types of Attributes (contd.,)

- **Single-value attribute** – Single-value attributes **contain single value**. For example – Social_Security_Number.
- **Multi-value attribute** – Multi-value attributes may **contain more than one values**. For example, a person can have more than one phone number, email_address, etc.
- For example a teacher entity can have multiple subject values.



- These attribute types can come together in a way like –
- *simple single-valued attributes*
- *simple multi-valued attributes*
- *composite single-valued attributes*
- *composite multi-valued attributes*

Entity-Set and Keys

- **Key** is an attribute or collection of attributes that uniquely identifies an entity among entity set.
- For example, the **roll_number** of a student makes him/her identifiable among students.
- **Super Key** – A set of attributes (one or more) that collectively identifies an entity in an entity set.
- **Candidate Key** – A minimal super key is called a candidate key. An entity set may have more than one candidate key.
- **Primary Key** – A primary key is one of the candidate keys chosen by the database designer to uniquely identify the entity set.

Relationship

- The association among entities is called a relationship.
- For example, an employee **works_at** a department, a student **enrolls** in a course.
- Here, Works_at and Enrolls are called relationships.
- For example, the entity “**carpenter**” may be related to the **entity** “**table**” by the **relationship** “**builds**” or “**makes**”. Relationships are represented by diamond shapes and are labeled using verbs.

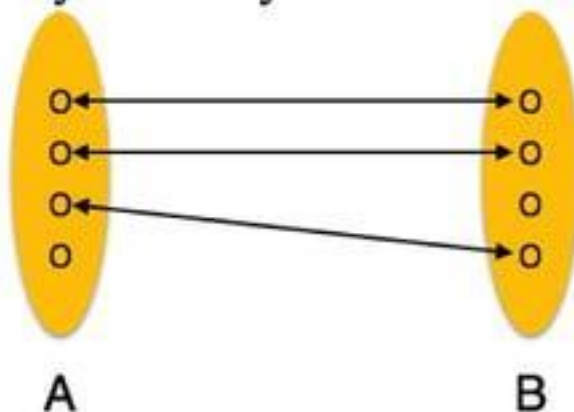


Degree of Relationship

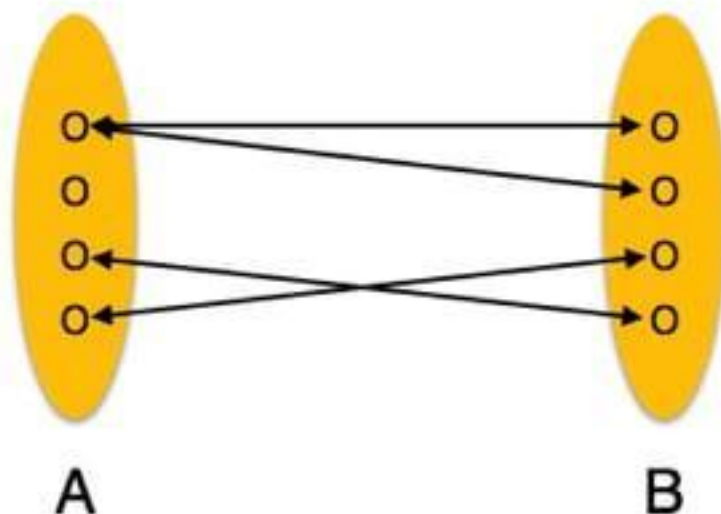
- The number of participating entities in a relationship defines the degree of the relationship.
- Binary = degree 2
- Ternary = degree 3
- n-ary = degree

Mapping Cardinalities

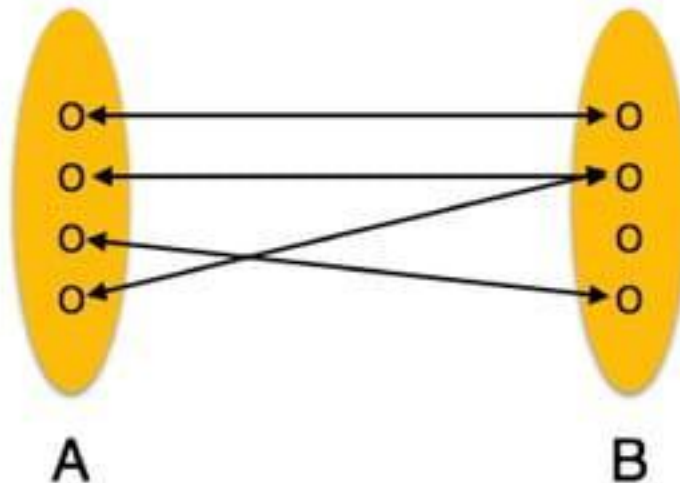
- **Cardinality** defines the number of entities in one entity set, which can be associated with the number of entities of other set via relationship set.
- **One-to-one** – One entity from entity set A can be associated with at most one entity of entity set B and vice versa.



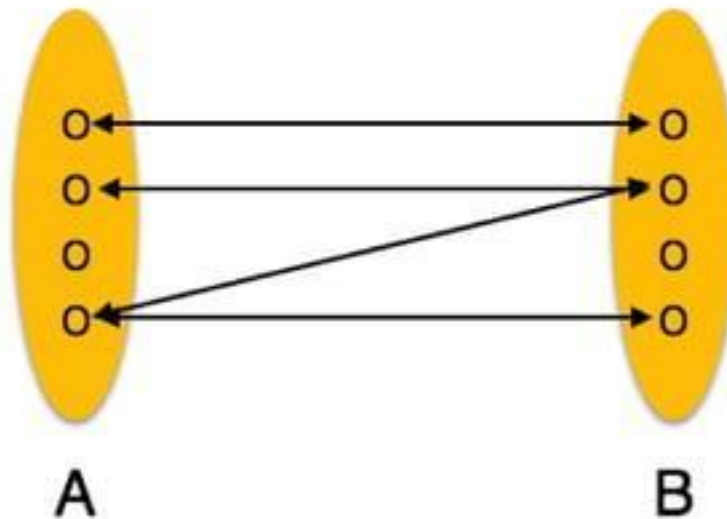
- **One-to-many** – One entity from entity set A can be associated with more than one entities of entity set B however an entity from entity set B, can be associated with at most one entity.



- **Many-to-one** – More than one entities from entity set A can be associated with at most one entity of entity set B, however an entity from entity set B can be associated with more than one entity from entity set A.



- **Many-to-many** – One entity from A can be associated with more than one entity from B and vice versa.



ER Diagram Representation

- Let us now learn how the ER Model is represented by means of an ER diagram.
- Any object, for example, entities, attributes of an entity, relationship sets, and attributes of relationship sets, can be represented with the help of an ER diagram.

Entity

- Entities are represented by means of rectangles. Rectangles are named with the entity set they represent.



Student



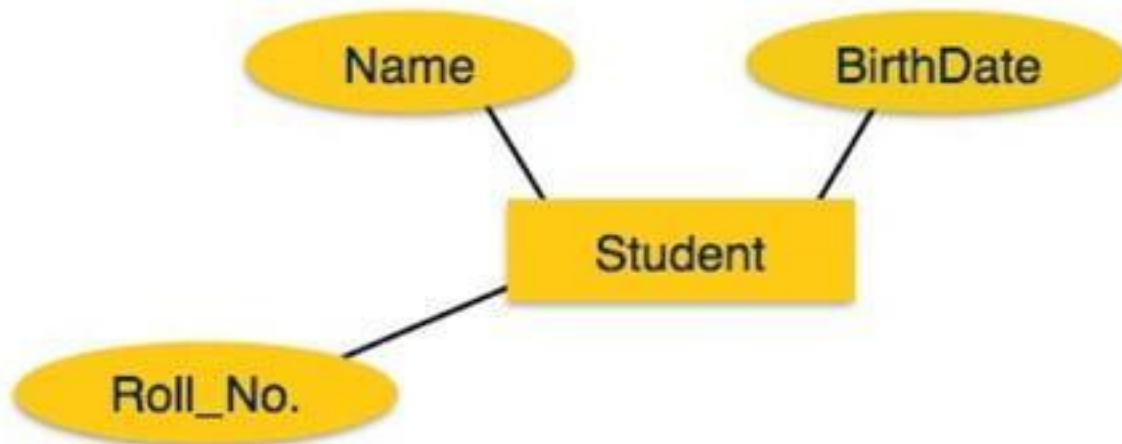
Teacher



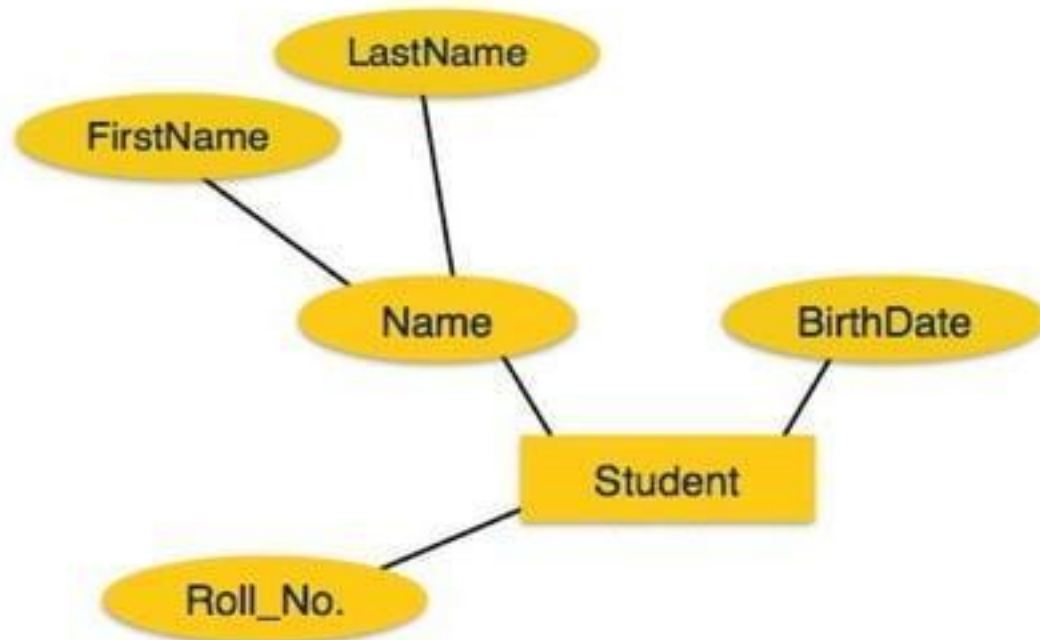
Projects

Attributes

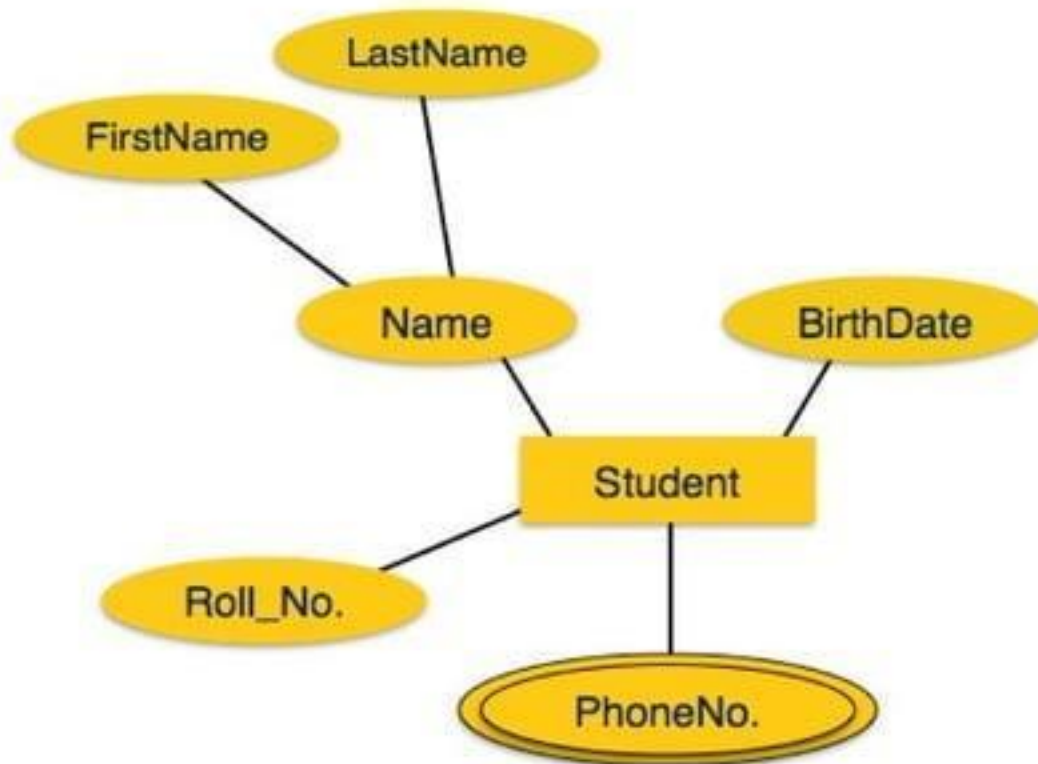
- Attributes are the properties of entities. Attributes are represented by means of ellipses.
- Every ellipse represents one attribute and is directly connected to its entity (rectangle).



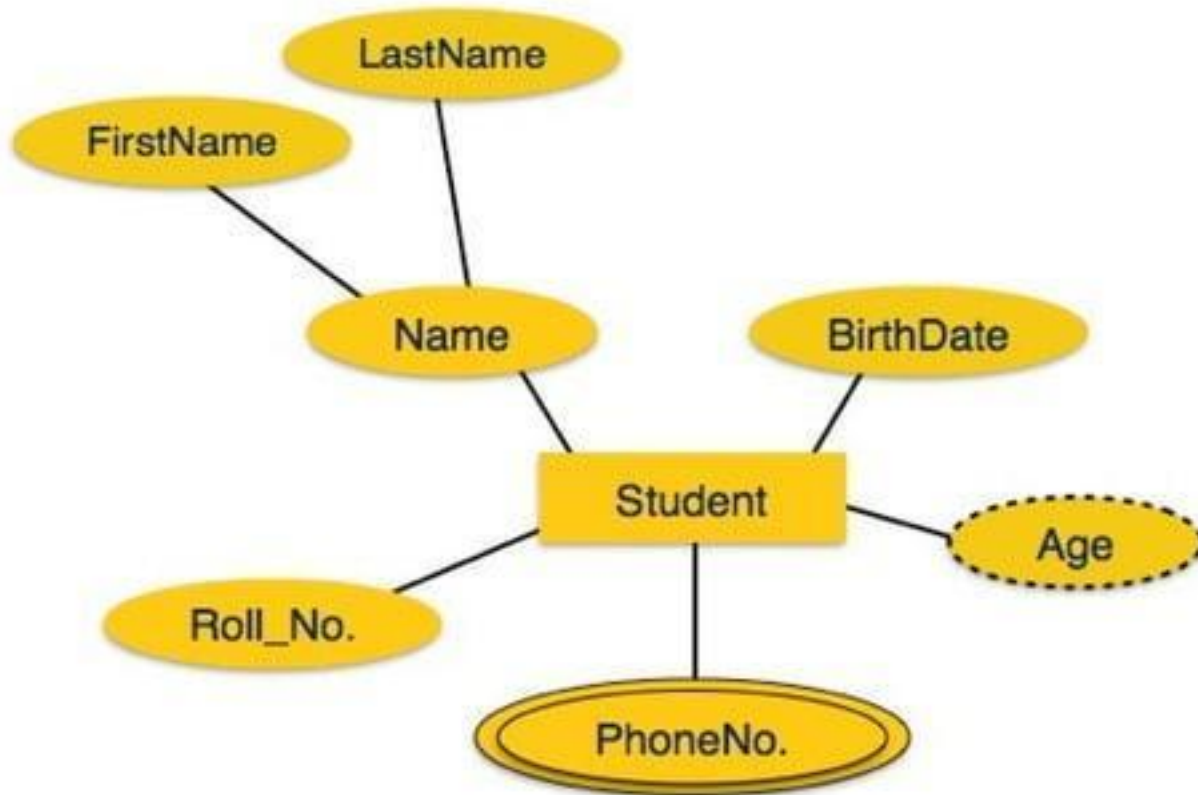
- If the attributes are **composite**, they are **further divided in a tree like structure**. Every node is then connected to its attribute. That is, composite attributes are represented by ellipses that are connected with an ellipse.



- **Multivalued** attributes are depicted by double ellipse.



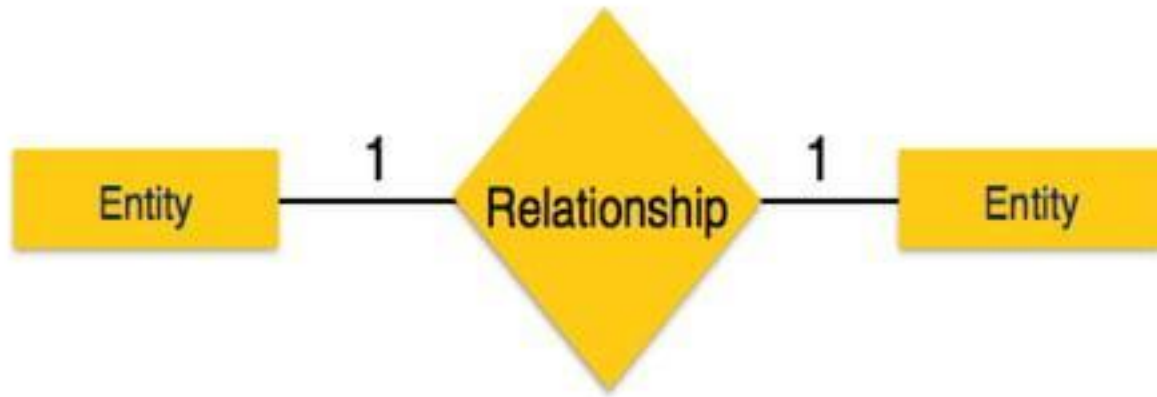
- **Derived** attributes are depicted by dashed ellipse.



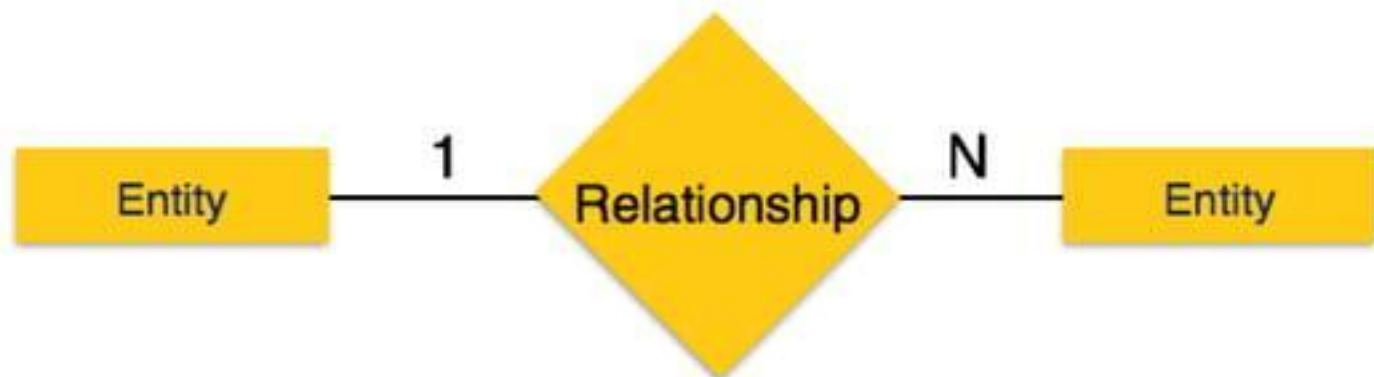
Relationship

- Relationships are represented by **diamond-shaped box**.
- Name of the relationship is written inside the diamond-box.
- All the entities (rectangles) participating in a relationship, are connected to it by a line.
- Binary Relationship and Cardinality
- A relationship where two entities are participating is called a **binary relationship**. Cardinality is the number of instance of an entity from a relation that can be associated with the relation.

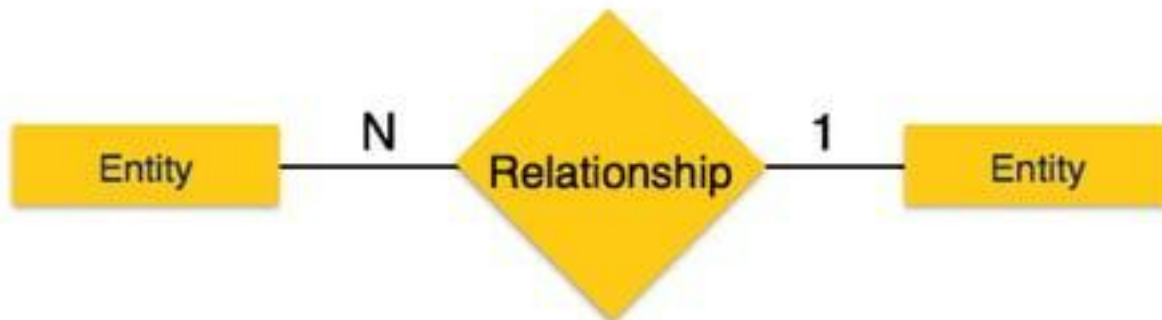
- **One-to-one** – When only one instance of an entity is associated with the relationship, it is marked as '1:1'. The following image reflects that only one instance of each entity should be associated with the relationship. It depicts one-to-one relationship.



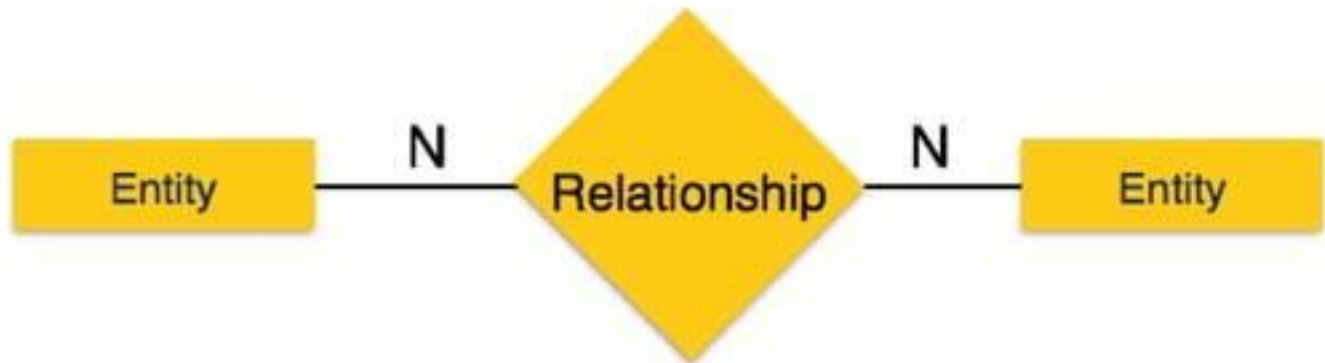
- **One-to-many** – When more than one instance of an entity is associated with a relationship, it is marked as '**1:N**'. The following image reflects that only *one instance of entity on the left and more than one instance of an entity on the right can be associated with the relationship*. It depicts one-to-many relationship.



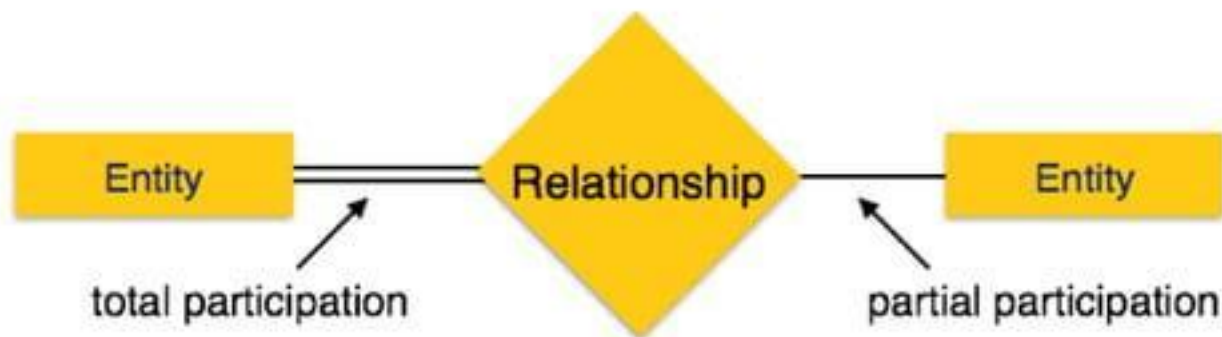
- **Many-to-one** – When *more than one instance of entity is associated with the relationship*, it is marked as 'N:1'. The following image reflects that more than one instance of an entity on the left and only one instance of an entity on the right can be associated with the relationship. It depicts many-to-one relationship.



- **Many-to-many** – The following image reflects that *more than one instance of an entity on the left and more than one instance of an entity on the right can be associated with the relationship*. It depicts many-to-many relationship.



- Participation Constraints
- **Total Participation** – Each entity is involved in the relationship. Total participation is represented by double lines.
- **Partial participation** – Not all entities are involved in the relationship. Partial participation is represented by single lines.

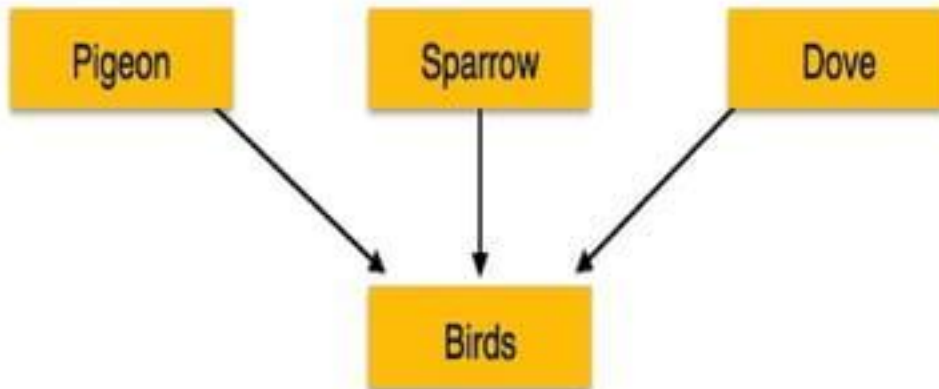


Generalization Aggregation

- The ER Model has the power of expressing database entities in a conceptual *hierarchical manner*.
- As the hierarchy *goes up*, it *generalizes the view of entities*, and as we **go deep** in the hierarchy, *it gives us the detail of every entity included*.
- Going up in this structure is called **generalization**, where entities are clubbed together to represent a more generalized view.
- For example, a particular student named Mira can be generalized along with all the students.
- The entity shall be a student, and further, the student is a person. The reverse is called **specialization** where a person is a student, and that student is Mira.

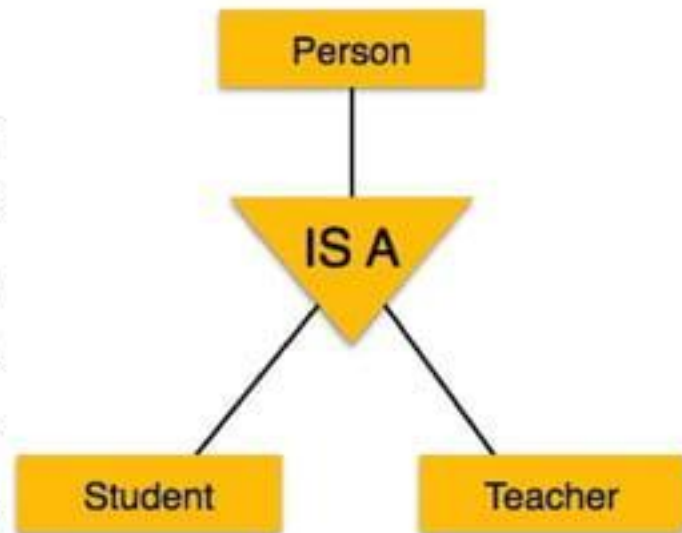
Generalization

- The process of generalizing entities, where the generalized entities contain the properties of all the generalized entities, is called generalization.
- In generalization, a number of entities are brought together into one generalized entity based on their similar characteristics. For example, pigeon, house sparrow, crow and dove can all be generalized as Birds.



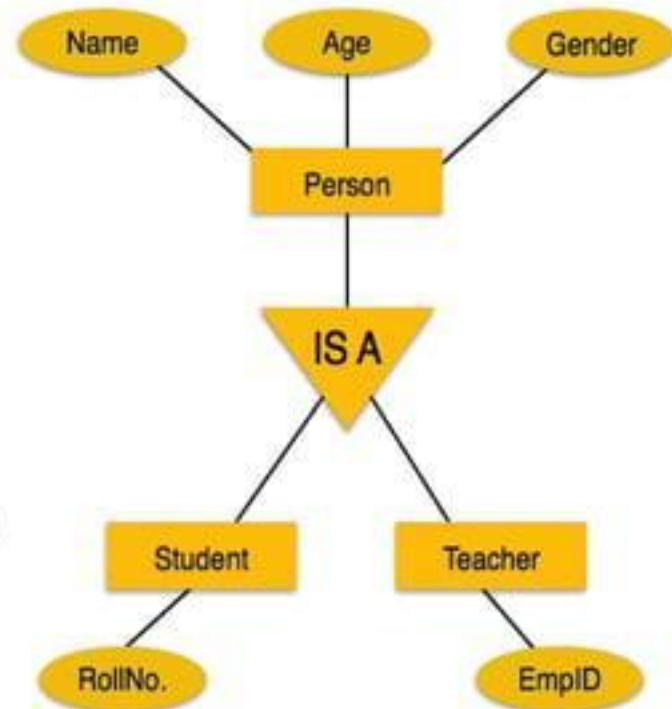
Specialization

- Specialization is the opposite of generalization. In specialization, a group of entities is divided into sub-groups based on their characteristics.
- Take a group '**Person**' for example. A person has **name, date of birth, gender, etc.** These properties are common in all persons, human beings. But in a **company**, persons can be identified as **employee, employer, customer, or vendor**, based on what role they play in the company.
- Similarly, in a **school database**, persons can be specialized as **teacher, student, or a staff**, based on what role they play in school as entities.



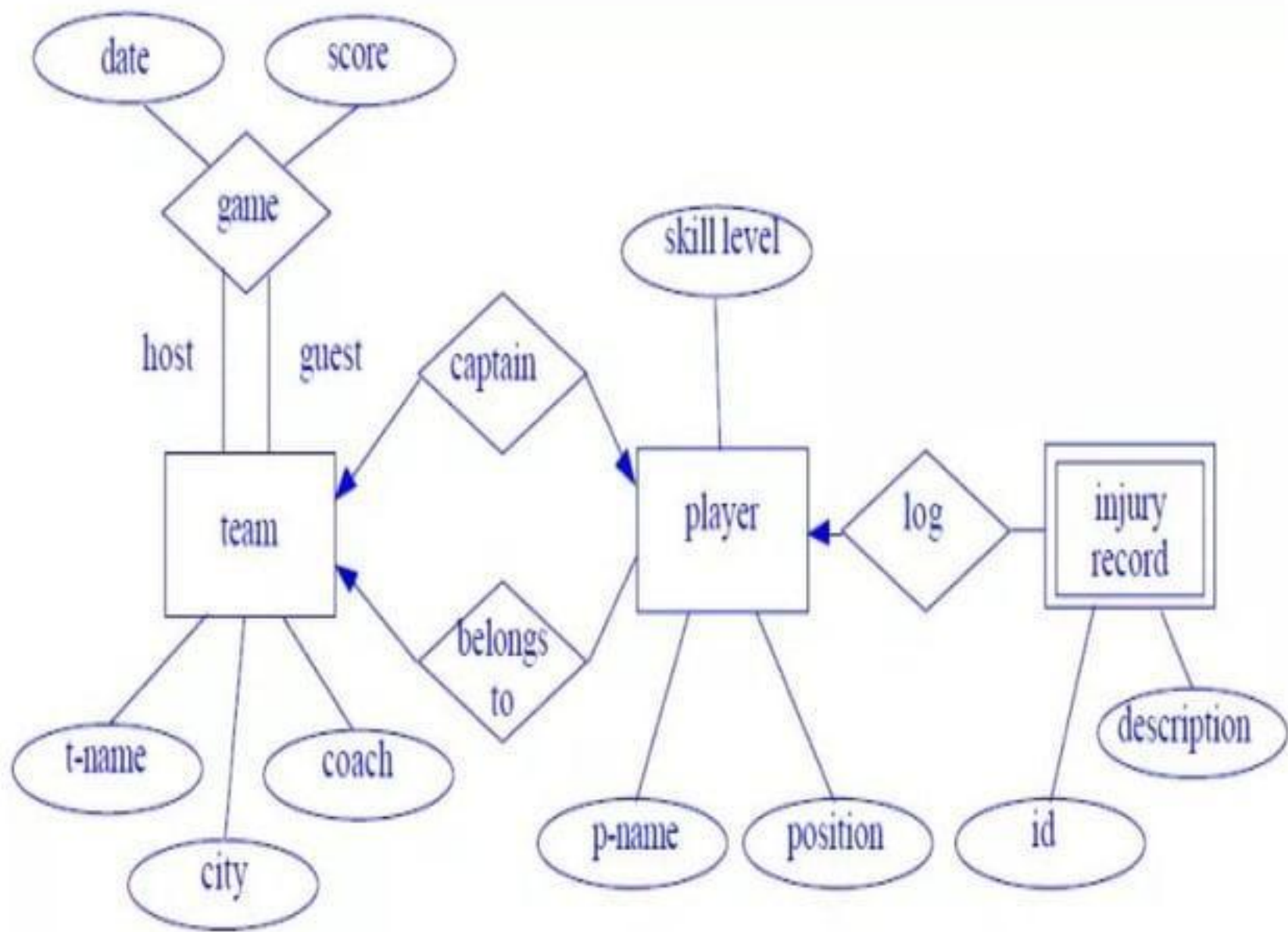
Inheritance

- We use all the above features of ER-Model in order to create classes of objects in object-oriented programming.
- *The details of entities are generally hidden* from the user; this process known as **abstraction**.
- Inheritance is an important feature of *Generalization and Specialization*. It allows lower-level entities to inherit the attributes of higher-level entities.
- For example, the attributes of a Person class such as name, age, and gender can be inherited by lower-level entities such as Student or Teacher.

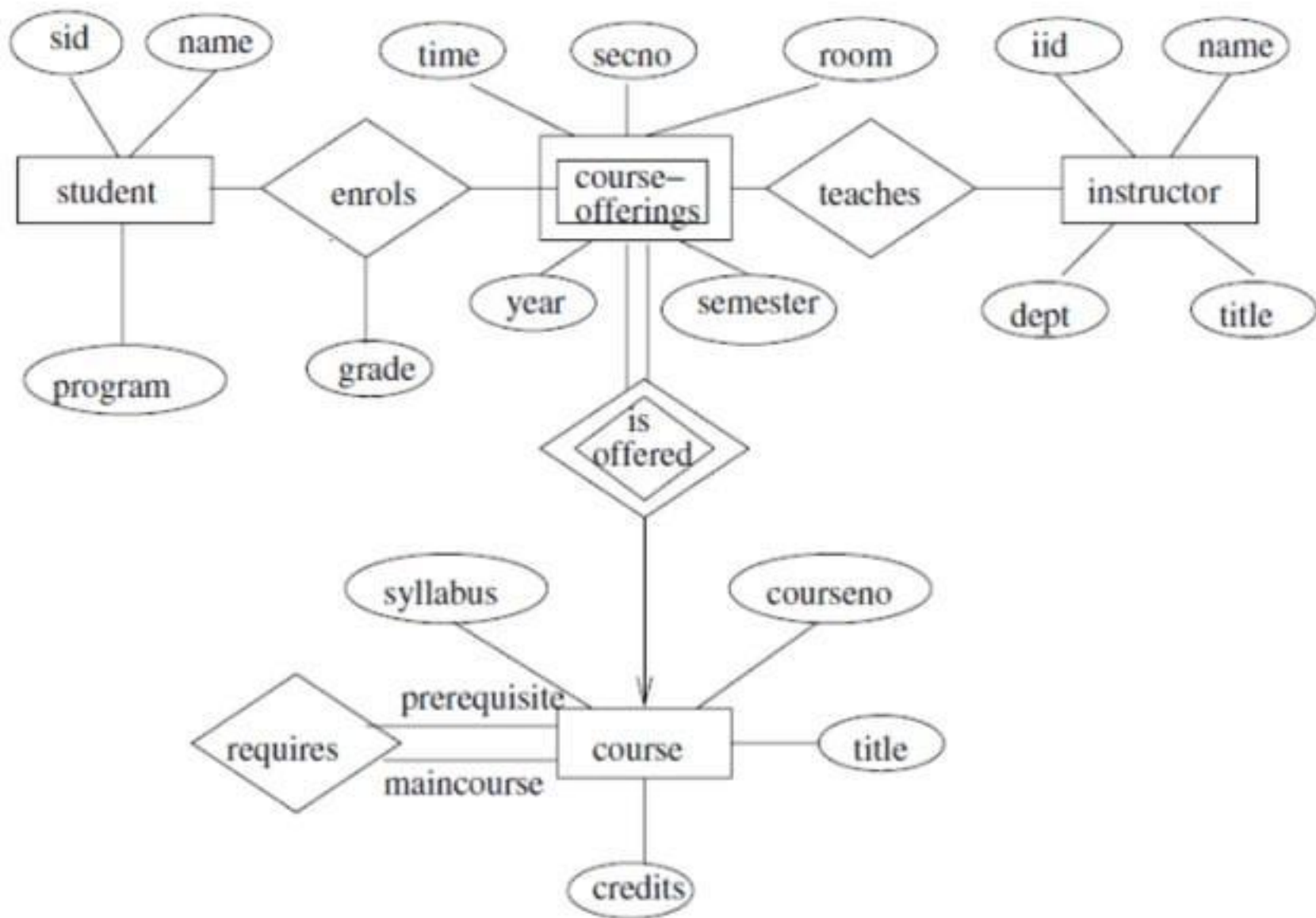


ER Diagram Example

- Suppose you are given the following requirements for a simple database for the National Hockey League (NHL):
- the NHL has many teams,
- each team has a name, a city, a coach, a captain, and a set of players,
- each player belongs to only one team,
- each player has a name, a position (such as left wing or goalie), a skill level, and a set of injury records,
- a team captain is also a player,
- a game is played between two teams (referred to as `host_team` and `guest_team`) and has a date (such as May 11th, 1999) and a score (such as 4 to 2).
- Construct a clean and concise ER diagram for the NHL database.



- **Question 2:**
- **A university registrar's office maintains data about the following entities:**
- **courses, including number, title, credits, syllabus, and prerequisites;**
- **course offerings, including course number, year, semester, section number, instructor(s), timings, and classroom;**
- **students, including student-id, name, and program;**
- **instructors, including identification number, name, department, and title.**
- **Further, the enrollment of students in courses and grades awarded to students in each course they are enrolled for must be appropriately modeled. Construct an E-R diagram for the registrar's office. Document all assumptions that you make about the mapping constraints.**

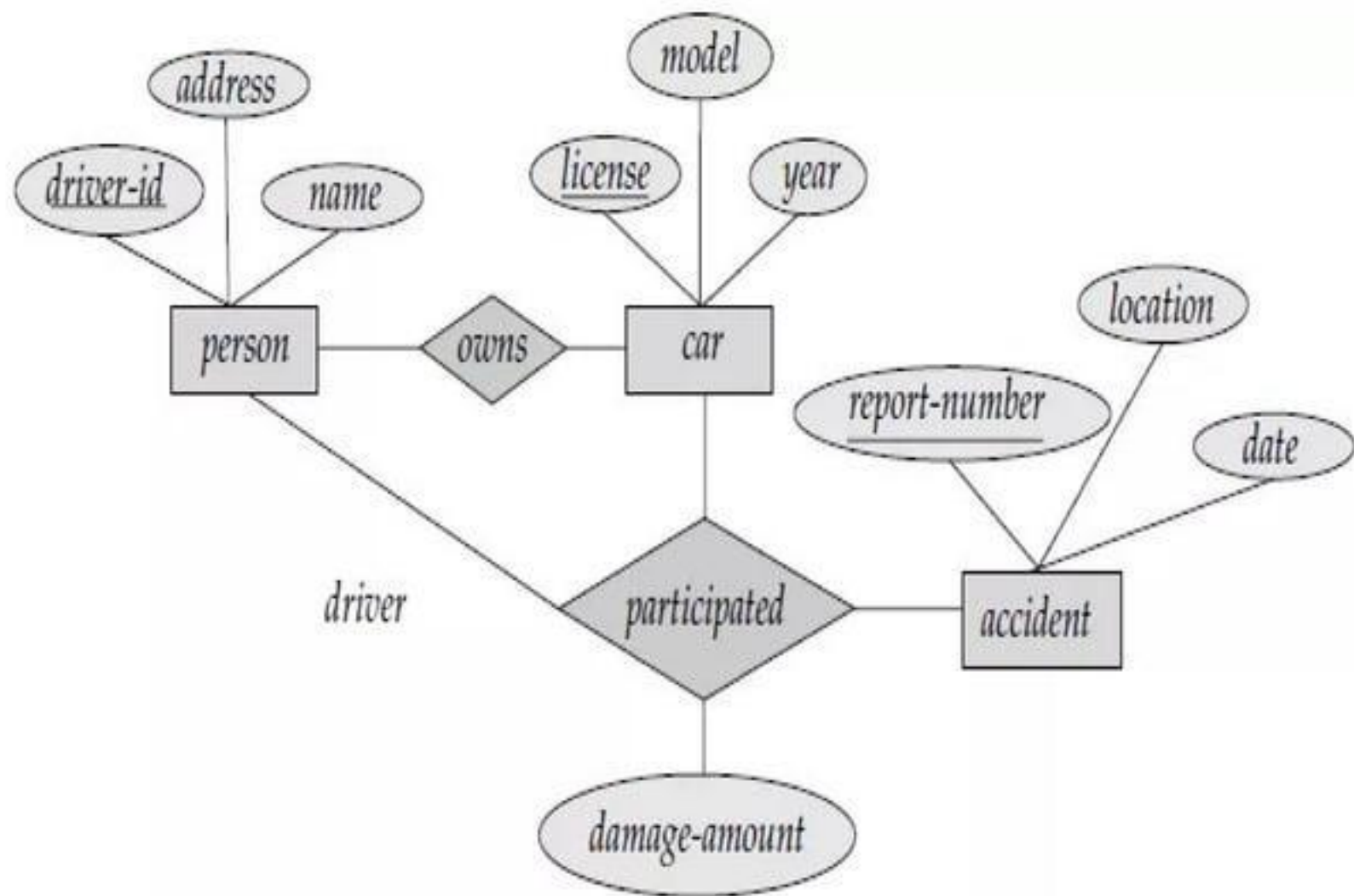


E-R diagram for a university.

- **Question 3:**
 - (a) Construct an E-R diagram for a car-insurance company whose customers own one or more cars each. Each car has associated with it zero to any number of recorded accidents.**
 - (b) Construct appropriate tables for the above ER Diagram ?**

Car insurance tables:

- person (driver-id, name, address)
- car (license, year, model)
- accident (report-number, date, location)
- participated(driver-id, license, report-number, damage-amount)



E-R diagram for a Car-insurance company.